Automatisation des installations avec Kickstart

1. Contexte et cycle de vie d'un serveur

Dans l'administration d'infrastructures de production, l'installation manuelle d'un système d'exploitation via une interface interactive présente plusieurs limites opérationnelles :

1. **Écarts de configuration :** Oubli d'une partition dédiée aux journaux système, mauvais choix de fuseau horaire, table de partitionnement hétérogène ou installation de paquets non requis.
2. **Temps de déploiement :** L'installation manuelle de plusieurs dizaines de serveurs physiques ou virtuels mobilise les équipes sur une longue durée.
3. **Reprise après sinistre :** En cas de défaillance matérielle, la reconstruction d'un nœud à l'identique doit pouvoir s'effectuer rapidement à partir d'un fichier de configuration versionné.

1.1 Découpage des phases d'industrialisation

Le déploiement d'un serveur en entreprise est scindé en deux étapes distinctes :

- **Le provisionnement initial (Kickstart / Preseed) :** Il prend en charge une machine vierge, configure le stockage (partitions, LVM, système de fichiers), installe le noyau et les paquets de base, configure l'interface réseau primaire et injecte les clés SSH d'administration. Son exécution est unique.
- **La gestion de configuration (Ansible / Puppet) :** Elle intervient une fois le système démarré et joignable par SSH pour déployer les services applicatifs, appliquer les règles de sécurité et maintenir le serveur dans le temps.

2. Fonctionnement de l'installateur Anaconda

Lorsqu'un serveur Rocky Linux démarre sur un support d'installation (ISO, clé USB ou réseau PXE), l'installateur **Anaconda** s'exécute en mémoire vive selon une séquence précise :

1. **Amorçage :** Le chargeur de démarrage (GRUB2 ou Syslinux) charge en mémoire le noyau Linux (`vmlinuz`) et le disque RAM initial (`initrd.img`).
2. **Initialisation :** `systemd` démarre dans l'environnement éphémère en mémoire (`initramfs`) et lance Anaconda.

3. **Lecture du Kickstart :** Anaconda récupère le fichier de réponses `ks.cfg` indiqué dans les paramètres de démarrage du noyau (`inst.ks=...`).
4. **Exécution séquentielle :**
    - Exécution des scripts de pré-installation (`%pre`) en mémoire vive.
    - Partitionnement des disques, création des volumes LVM et formatage des systèmes de fichiers montés sous `/mnt/sysimage`.
    - Résolution des dépendances DNF/RPM et installation des paquets.
    - Exécution des scripts de post-installation (`%post`) directement dans l'environnement du nouveau système (`chroot /mnt/sysimage`).
    - Installation du chargeur GRUB2 sur le disque et redémarrage.

2.1 Consoles virtuelles de diagnostic et fichier modèle

Pendant le déroulement de l'installation, Anaconda met à disposition plusieurs terminaux virtuels accessibles via `Ctrl+Alt+F1` à `F5` (ou via le multiplexeur `tmux`) :

- **`tty1` :** Écran principal de suivi d'installation.
- **`tty2` :** Shell interactif `root` permettant d'inspecter les disques (`lsblk`), le réseau (`ip a`) ou de tester des commandes à chaud.
- **`tty3` :** Journal général d'installation (`/tmp/anaconda.log`).
- **`tty4` :** Journal dédié au stockage et au partitionnement (`/tmp/storage.log`).
- **`tty5` :** Sortie standard et erreurs des scripts `%pre` et `%post` (`/tmp/program.log`).

Note sur le fichier modèle : À la fin de toute installation de Rocky Linux (même réalisée manuellement), Anaconda génère le fichier **`/root/anaconda-ks.cfg`**. Ce fichier retrace les choix effectués lors de l'installation et sert de base de travail pour industrialiser les déploiements suivants.

3. Structure d'un fichier Kickstart (`ks.cfg`)

Un fichier Kickstart est un fichier texte composé de quatre sections : les directives principales (système, réseau, stockage), la sélection des paquets (`%packages`), les scripts de pré-installation (`%pre`) et les scripts de post-installation (`%post`).

3.1 Directives système, réseau et authentification

Les directives d'en-tête définissent le comportement de l'installateur, la localisation et les comptes d'accès initiaux :

```
None
# Mode d'installation en console texte et redémarrage automatique en fin de
processus
text
reboot --eject

# Localisation, disposition clavier et fuseau horaire (horloge matérielle sur UTC)
lang fr_FR.UTF-8
keyboard --vckeymap=fr --xlayouts='fr'
timezone Europe/Paris --utc

# Source d'installation (Média optique local ou dépôt HTTP distant)
cdrom
# url --url="http://192.168.4.100/repo/rocky9/BaseOS/"

# Sécurité : activation de SELinux et du pare-feu local avec ouverture du port SSH
selinux --enforcing
firewall --enabled --service=ssh

# Configuration réseau par défaut en DHCP
network --bootproto=dhcp --device=link --activate --onboot=yes
--hostname=srv-rocky.infra.lan

# Exemple de configuration réseau en IP statique :
# network --bootproto=static --ip=192.168.4.101 --netmask=255.255.255.0
--gateway=192.168.4.1 --nameserver=192.168.4.1 --device=link --activate

# Définition du mot de passe root chiffré en SHA-512
rootpw --iscrypted $6$SelAleatoire$HashSHA512GenereParOpenSSL...

# Création d'un compte d'administration membre du groupe wheel avec clé publique
SSH
user --name=adminsys --groups=wheel --iscrypted
--password=$6$SelAleatoire$HashSHA512...
sshkey --username=adminsys "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI...
adminsys@master"
```

Pour éviter d'inscrire des mots de passe en clair dans le fichier Kickstart, le condensat SHA-512 est généré en amont sur le poste d'administration avec la commande suivante :

```
None
openssl passwd -6 -salt "Sel2026" "MotDePasseRobuste"
```

3.2 Automatisation du stockage et partitionnement LVM

En environnement de production, l'utilisation de **LVM (*Logical Volume Manager*)** associée au système de fichiers **XFS** permet de redimensionner les volumes logiques à chaud et d'isoler les répertoires critiques (`/var`, `/var/log`, `/tmp`). Cette séparation évite qu'une saturation des journaux ou des fichiers temporaires ne bloque la partition racine `/`.

```
None
# 1. Effacement des tables de partitions existantes et initialisation du disque
sda
zerombr
clearpart --all --initlabel --drives=sda

# 2. Installation du chargeur de démarrage GRUB2 dans le secteur d'amorçage
bootloader --location=mbr --boot-drive=sda --append="crashkernel=auto"

# 3. Partition standard /boot (hors LVM)
part /boot --fstype="xfs" --ondisk=sda --size=1024

# 4. Création du Volume Physique (PV) LVM occupant tout l'espace restant (--grow)
part pv.01 --fstype="lvmpv" --ondisk=sda --size=10240 --grow

# 5. Création du Groupe de Volumes (VG) nommé 'vg_system' (taille d'extent = 4 Mo)
volgroup vg_system --pesize=4096 pv.01

# 6. Création des Volumes Logiques (LV) avec options de montage sécurisées sur
/tmp
logvol swap  --fstype="swap" --name=lv_swap --vgname=vg_system --size=2048
logvol /     --fstype="xfs"  --name=lv_root --vgname=vg_system --size=8192
logvol /var  --fstype="xfs"  --name=lv_var  --vgname=vg_system --size=4096
logvol /tmp  --fstype="xfs"  --name=lv_tmp  --vgname=vg_system --size=1024
--fsoptions="nodev,nosuid,noexec"
```

3.3 Sélection des paquets (`%packages`)

La section `%packages` définit l'environnement de base et les utilitaires requis dès le premier démarrage. Le préfixe `@^` désigne un environnement complet, `@` désigne un groupe de paquets, et `-` permet d'exclure un paquet :

```
None
%packages
@^minimal-environment
vim-enhanced
curl
bash-completion
```

```
python3
-iwl*
-aic94xx-firmware
%end
```

4. Personnalisation dynamique : Les blocs `%pre` et `%post`

Un fichier Kickstart purement statique échoue lorsque le parc matériel présente des contrôleurs de stockage différents : un serveur SAS présente un disque `/dev/sda`, un serveur NVMe expose `/dev/nvme0n1` et une machine virtuelle KVM utilise `/dev/vda`.

4.1 Le script de pré-installation (`%pre`)

Le bloc `%pre` s'exécute en mémoire vive avant l'analyse du partitionnement. Associé à la directive **`%include`**, il permet d'inspecter le matériel de la machine cible pour générer dynamiquement la configuration de stockage :

```
None
# Inclusion du fichier de partitionnement qui sera généré par le bloc %pre
%include /tmp/part-include.cfg

%pre --interpreter=/bin/bash --log=/tmp/ks-pre.log
# 1. Identification automatique du premier disque disponible
FIRST_DISK=$(lsblk -ndo NAME,TYPE | awk '$2=="disk" {print $1; exit}')

# 2. Calcul de la mémoire vive totale en Mo pour adapter la taille du Swap
MEM_MB=$(awk '/MemTotal/ {printf "%.0f", $2/1024}' /proc/meminfo)
if [ "$MEM_MB" -le 4096 ]; then
    SWAP_SIZE=2048
else
    SWAP_SIZE=4096
fi

# 3. Génération du fichier de configuration du stockage
cat << EOF > /tmp/part-include.cfg
zerombr
clearpart --all --initlabel --drives=${FIRST_DISK}
bootloader --location=mbr --boot-drive=${FIRST_DISK}
part /boot --fstype="xfs" --ondisk=${FIRST_DISK} --size=1024
part pv.01 --fstype="lvmpv" --ondisk=${FIRST_DISK} --size=8192 --grow
volgroup vg_sys pv.01
logvol swap --fstype="swap" --name=lv_swap --vgname=vg_sys --size=${SWAP_SIZE}
logvol /    --fstype="xfs"  --name=lv_root --vgname=vg_sys --size=6144 --grow
EOF
```

```
%end
```

4.2 Le script de post-installation (`%post`)

Le bloc `%post` s'exécute après l'installation des paquets, directement à l'intérieur du nouveau système de fichiers (`chroot /mnt/sysimage`). Il permet d'appliquer les réglages finaux d'administration :

```
None
%post --interpreter=/bin/bash --log=/root/ks-post.log
# 1. Autorisation d'élévation de privilèges sudo sans mot de passe pour le groupe
wheel
echo "%wheel ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/10-wheel-nopasswd
chmod 0440 /etc/sudoers.d/10-wheel-nopasswd

# 2. Création d'une bannière d'avertissement SSH
cat << 'EOF' > /etc/issue.net
Acces reserve aux administrateurs autorises.
Toutes les actions effectuees sur ce serveur sont journalisees.
EOF

# 3. Enregistrement d'un marqueur d'installation
echo "Provisioned via Kickstart on $(date -u +"%Y-%m-%dT%H:%M:%SZ")" >
/etc/provisioned.info
%end
```

5. Validation syntaxique avec `ksvalidator`

Une erreur de syntaxe dans un fichier Kickstart interrompt l'automatisation et bloque le serveur sur une invite interactive pendant l'amorçage. Sous Rocky Linux, le paquet **`pykickstart`** fournit l'outil **`ksvalidator`**, qui permet de contrôler la conformité du fichier avant sa mise en production :

```
None
# Installation de l'utilitaire de validation
sudo dnf install -y pykickstart

# Vérification de la syntaxe selon la grammaire RHEL 9 / Rocky Linux 9
ksvalidator -v RHEL9 /chemin/vers/ks.cfg
```

Si la commande ne retourne aucune erreur et que le code de sortie (`echo $?`) vaut `0`, le fichier Kickstart est syntaxiquement valide.

Synthèse : Directives et outils Kickstart

Directive / Outil	**Rôle et utilisation**
`/root/anaconda-ks.cfg`	Fichier de référence généré automatiquement par Anaconda après l'installation du système.
`ksvalidator -v RHEL9`	Vérifie la conformité syntaxique d'un fichier Kickstart avant son déploiement (paquet `pykickstart`).
`openssl passwd -6`	Génère un condensat de mot de passe SHA-512 pour les directives `rootpw` et `user`.
`%include /tmp/fichier`	Charge des directives Kickstart générées dynamiquement lors de la phase `%pre`.
`%pre`	Script exécuté en mémoire vive (`initramfs`) avant le partitionnement des disques.
`%post`	Script exécuté dans un environnement `chroot` (`/mnt/sysimage`) après l'installation des paquets.